



EC-220 Computer System Architecture

Dr. Ahsan Shahzad

Department of Computer and Software Engineering (DC&SE)

College of Electrical and Mechanical Engineering (CEME)

National University of Science and Technology (NUST)

Week # 7b_Corona Break



Multiplication

- Easy in binary
 - If multiplier digit is 1, copy multiplicand and shift by as many bits as the digit position
 - Otherwise copy zeros and shift by as many bits as the digit position
- Multiplication of **m-bit by n-bit** requires up to **n+m bits** to store the result
- Need to cope with overflow
- Need sophisticated hardware

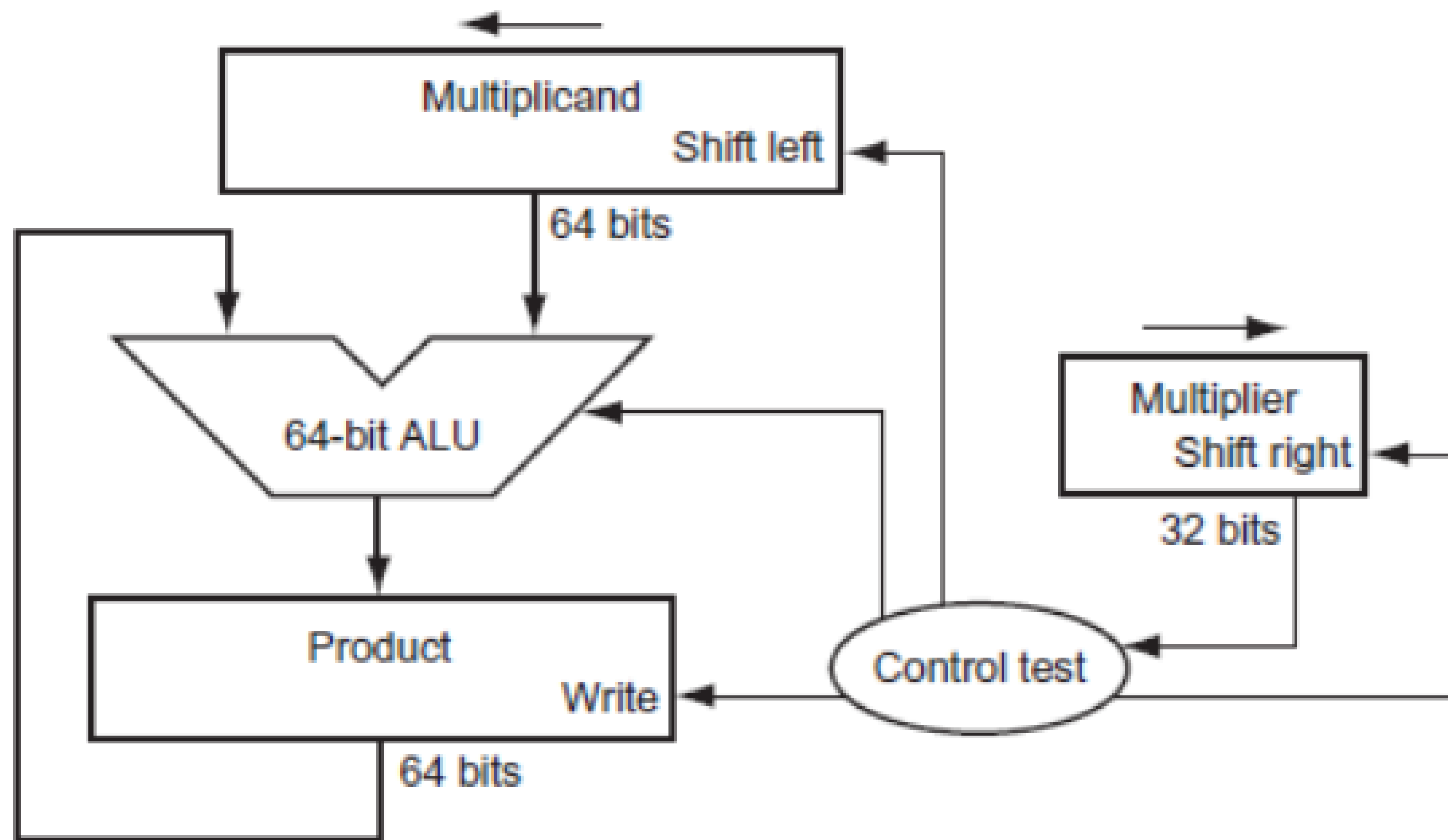
Multiplicand		1000 _{ten}
Multiplier	x	1001 _{ten}
		1000
		0000
		0000
		1000
Product		
		1001000 _{ten}

Implementing Multiplication in Hardware.

- 32-bit multiplicand
 - It may be shifted up to 32 times to the left
 - Need 64-bit register to hold multiplicand
 - 0s in the leftmost 32 bits
- 32-bit multiplier
 - Needs to be shifted right one bit
 - Shifted out bit needs to be checked to see whether we need to add multiplicand or not in the result
- 64-bit product
 - 2 32-bit operands

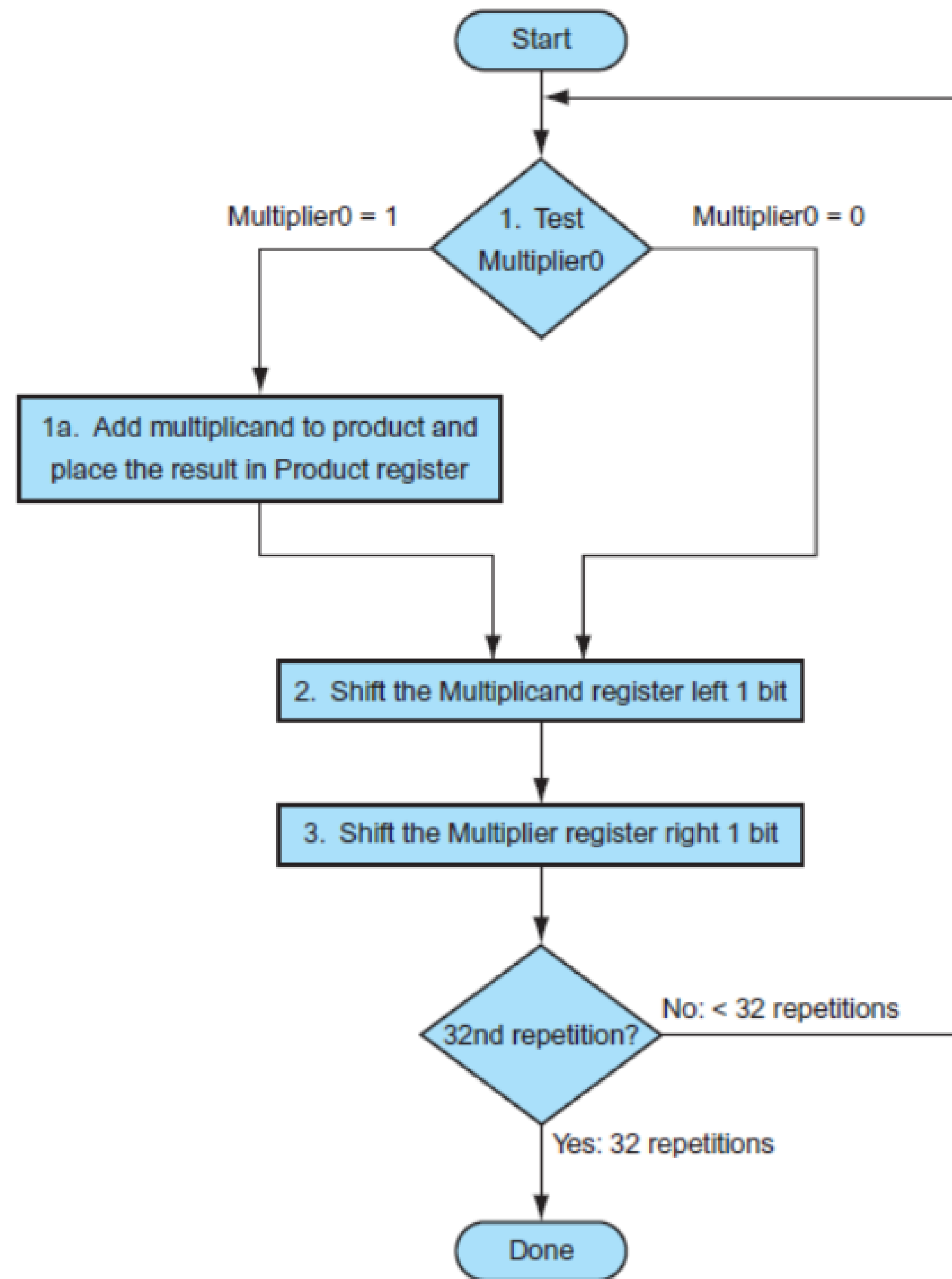
Multiplication Hardware

- First version



First Multiplication Algorithm

- **Three basic steps for each bit**
 - Multiplicand addition
 - Left shift
 - Right shift
- **32 repetitions to get product**
- **Require almost 100 clock cycles**
- **Multiplication is Slow**
 - Avoid as much as possible, use shifting if multiplying with powers of 2.



Example: Multiply Algorithm

- Multiply example using previous algorithm

Iteration	Step	Multiplier	Multiplicand	Product
0	Initial values	001 <u>1</u>	0000 0010	0000 0000
1	1a: $1 \Rightarrow \text{Prod} = \text{Prod} + \text{Mcand}$	0011	0000 0010	0000 0010
	2: Shift left Multiplicand	0011	0000 0100	0000 0010
	3: Shift right Multiplier	000 <u>1</u>	0000 0100	0000 0010
2	1a: $1 \Rightarrow \text{Prod} = \text{Prod} + \text{Mcand}$	0001	0000 0100	0000 0110
	2: Shift left Multiplicand	0001	0000 1000	0000 0110
	3: Shift right Multiplier	000 <u>0</u>	0000 1000	0000 0110
3	1: $0 \Rightarrow$ No operation	0000	0000 1000	0000 0110
	2: Shift left Multiplicand	0000	0001 0000	0000 0110
	3: Shift right Multiplier	000 <u>0</u>	0001 0000	0000 0110
4	1: $0 \Rightarrow$ No operation	0000	0001 0000	0000 0110
	2: Shift left Multiplicand	0000	0010 0000	0000 0110
	3: Shift right Multiplier	0000	0010 0000	0000 0110

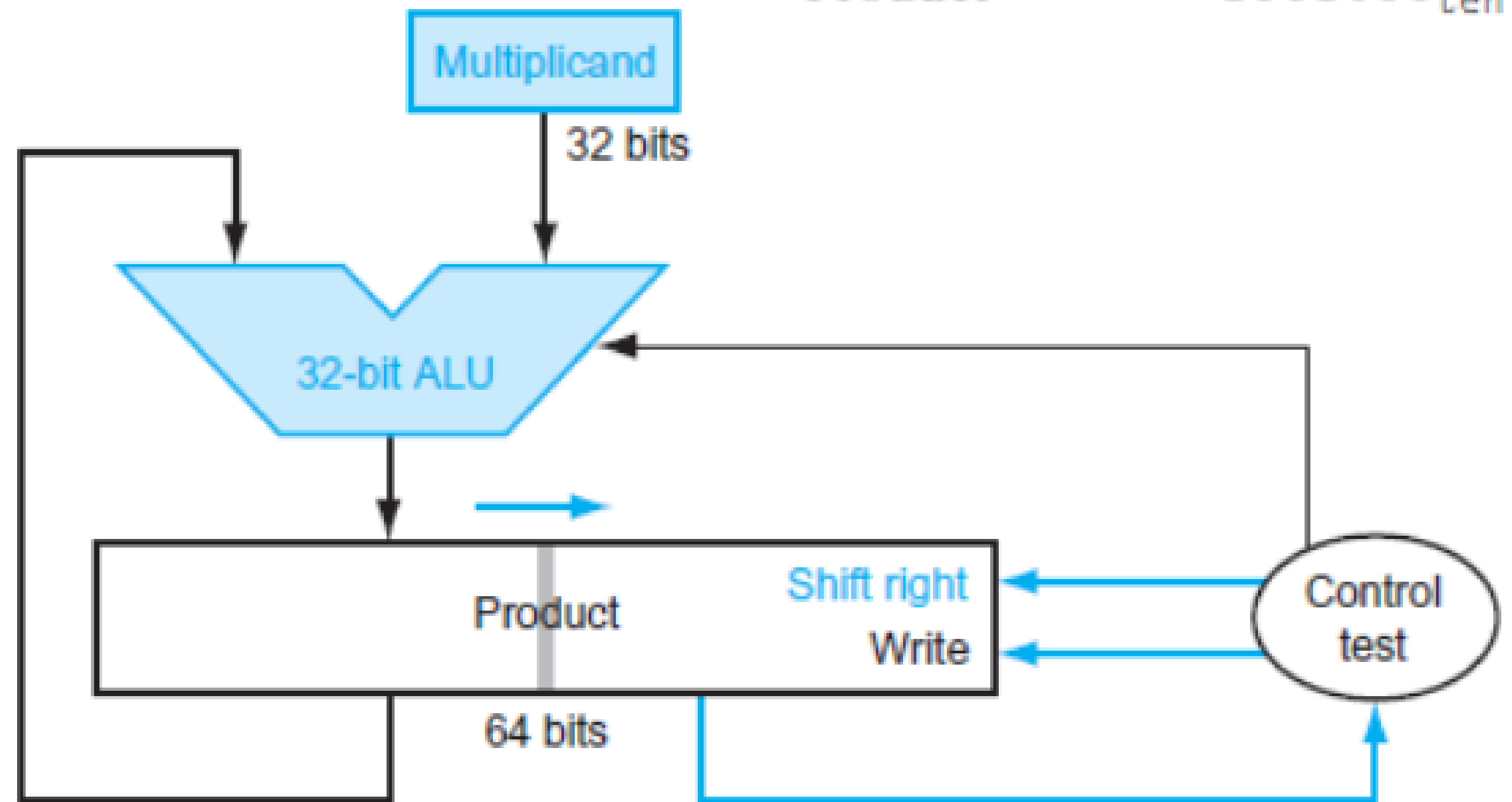
FIGURE 3.6 Multiply example using algorithm in Figure 3.4. The bit examined to determine the next step is circled in color.

Hardware Multiplier – Refined Version

- Partial sum stored in upper 32 bits
- Multiplier in lower 32 bits
- Multiplier shifted out bit by bit
- Partial sum shifts in place of multiplier
- Do not need to extend multiplicand!
- Everything happens in place
- Faster, simpler hardware!

$$\begin{array}{r} \text{Multiplicand} \\ \text{Multiplier} \end{array} \times \begin{array}{r} 1000_{\text{ten}} \\ 1001_{\text{ten}} \\ \hline 1000 \\ 0000 \\ 0000 \\ 1000 \\ \hline 1001000_{\text{ten}} \end{array}$$

Product

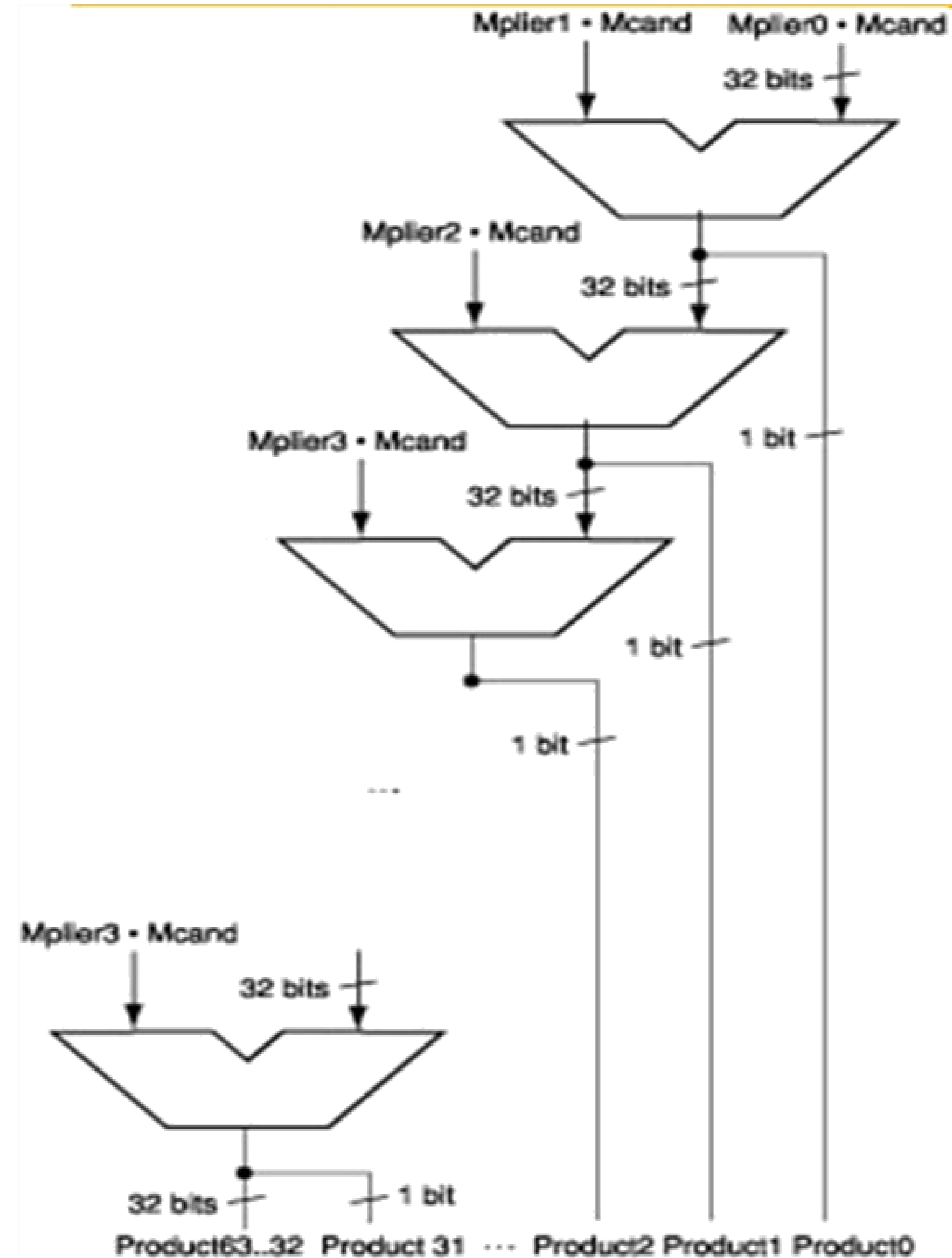


Signed Multiplication

- **Convert operands to positive numbers**
- **Remember original digits**
- **Multiply using original algorithm for 31 iterations**
- **Then add new sign**
 - Caution: shifting the product sum needs to sign extend the product sign

Faster Multiplication

- Uses multiple adders
- Cost / Performance tradeoff
- Can be pipelined



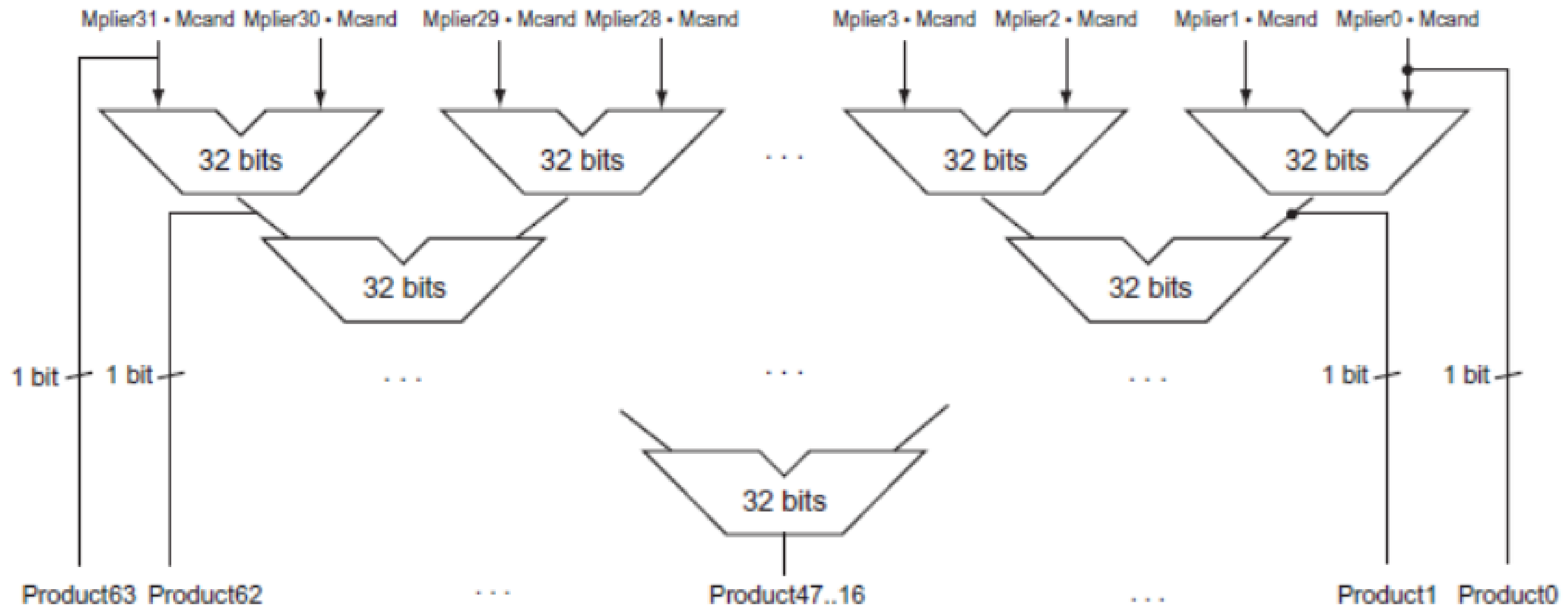
Faster Multiplication

Why is the multiplier with 32 adders faster?

- **Original multiplier needs a clock per bit**
- **Optimized multiplier needs:**
 - An and operation (parallel)
 - 32 adders (pipelined)
 - Each adder receives output of previous multiplier
 - Least significant bit goes to result
 - 31 upper bits plus carry out go to next multiplier
- **Pipeline latency:**
 - Cost of first multiply slow
 - Following multiplies fast (presumably one per cycle)

Even Faster Multiplication

- An alternative way
 - Parallel tree organization
- $\log_2(32) \rightarrow 5$
- **Multiplication time:** From 32 to 5 (32-bit) add times.



Multiplication in MIPS

- *Instructions:* **mult, multu**
- *Registers:* **Hi – Lo**, act as an extended 64 bit register
- *Instructions:*
 - **mflo** (move from Lo), to bring the product (result) into registers
 - **mfhi** (move from hi), to bring the product (result) into registers
- **Overflow Checking**
 - No automatic check, both instructions *mult* and *multu* ignore overflow.
 - Need code to test
 - If Hi = 0 (*multu*) or replicated sign of Lo (*mult*) -> NO OVERFLOW
 - Use mfhi and test the above condition